

```

#include "ArrayGenerator.h"
#include "SortTester.h"
#include <windows.h>
#include <iostream>
#include <vector>

using std::cin;
using std::cout;

int main() {
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    ArrayGenerator gen;
    std::vector<int> sizes;
    for (int n = 500; n <= 100000; n += 100) {
        sizes.push_back(n);
    }
    std::vector<std::string> algorithms = {"merge", "hybrid"};
    std::vector<std::string> arrayTypes = {"random", "reversed", "nearly_sorted"};
    auto randomMax = gen.randomArray(100000);
    auto reversedMax = gen.reversedArray(100000);
    auto nearlyMax = gen.nearlySortedArray(100000, 10);
    for (int n: sizes) {
        std::vector<std::vector<int> > arrays = {
            std::vector<int>(randomMax.begin(), randomMax.begin() + n),
            std::vector<int>(reversedMax.begin(), reversedMax.begin() + n),
            std::vector<int>(nearlyMax.begin(), nearlyMax.begin() + n)
        };
        for (int i = 0; i < arrays.size(); ++i) {
            for (const auto &algo: algorithms) {
                long long t = SortTester::measureTime(arrays[i], algo);
                SortTester::saveToCSV(arrayTypes[i], algo, n, t);
            }
        }
    }
    return 0;
}

```

```

#ifndef ARRAY_GENERATOR_H
#define ARRAY_GENERATOR_H

#include <vector>
#include <algorithm>
#include <cstdlib>
#include <ctime>

class ArrayGenerator {
public:

```

```

ArrayGenerator() { std::srand(std::time(nullptr)); }

std::vector<int> randomArray(int n, int maxValue = 10000) {
    std::vector<int> arr(n);
    for (int i = 0; i < n; ++i) {
        arr[i] = std::rand() % (maxValue + 1);
    }
    return arr;
}

std::vector<int> reversedArray(int n, int maxValue = 10000) {
    auto arr = randomArray(n, maxValue);
    std::sort(arr.begin(), arr.end(), std::greater<int>());
    return arr;
}

std::vector<int> nearlySortedArray(int n, int swaps = 10) {
    auto arr = randomArray(n);
    std::sort(arr.begin(), arr.end());
    for (int i = 0; i < swaps; ++i) {
        int a = std::rand() % n;
        int b = std::rand() % n;
        std::swap(arr[a], arr[b]);
    }
    return arr;
}
};

#endif

```

```

#ifndef SORT_TESTER_H
#define SORT_TESTER_H

#include <vector>
#include <chrono>
#include <numeric>
#include <iostream>
#include <fstream>
#include <string>

using std::vector;

class SortTester {
public:
    static void insertionSort(vector<int> &arr, int left, int right) {
        for (int i = left + 1; i < right; ++i) {
            int key = arr[i];
            int j = i - 1;
            while (j >= left && arr[j] > key) {
                arr[j + 1] = arr[j];
                --j;
            }
            arr[j + 1] = key;
        }
    }
};

```

```

        }
        arr[j + 1] = key;
    }
}

static void merge(vector<int> &arr, int left, int mid, int right) {
    int n1 = mid - left, n2 = right - mid;
    vector<int> L(arr.begin() + left, arr.begin() + mid);
    vector<int> R(arr.begin() + mid, arr.begin() + right);
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}

static void mergeSort(vector<int> &arr, int left, int right) {
    if (right - left <= 1) {
        return;
    }
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid, right);
    merge(arr, left, mid, right);
}

static void hybridSortWithThreshold(vector<int> &arr, int left, int right, int
threshold) {
    if (right - left <= threshold) {
        insertionSort(arr, left, right);
        return;
    }
    int mid = left + (right - left) / 2;
    hybridSortWithThreshold(arr, left, mid, threshold);
    hybridSortWithThreshold(arr, mid, right, threshold);
    merge(arr, left, mid, right);
}

static long long measureTime(vector<int> arr, const std::string &algorithm =
"merge", int threshold = 15, int repetitions = 5) {
    vector<long long> times;
    for (int i = 0; i < repetitions; ++i) {
        vector<int> copy = arr;
        auto start = std::chrono::high_resolution_clock::now();
        if (algorithm == "merge") {

```

```

        mergeSort(copy, 0, static_cast<int>(copy.size()));
    } else if (algorithm == "hybrid") {
        hybridSortWithThreshold(copy, 0, static_cast<int>(copy.size()),
threshold);
    } else {
        throw std::invalid_argument("чет не то ввел" + algorithm);
    }
    auto elapsed = std::chrono::high_resolution_clock::now() - start;
    times.push_back(std::chrono::duration_cast<std::chrono::milliseconds>
(elapsed).count());
}
long long sum = std::accumulate(times.begin(), times.end(), 0LL);
return sum / repetitions;
}

static void saveToCSV(const std::string &arrayType, const std::string
&algorithm, int size, long long timeMs, const std::string &filename =
"result.csv") {
    std::ofstream out;
    out.open(filename, std::ios::app);
    out << "array_type,algorithm,size,time_ms\n";
    out << arrayType << "," << algorithm << "," << size << "," << timeMs <<
"\n";
    out.close();
}
};

#endif

```

ID своейсылки по задаче A2i в системе CodeForces с реализацией MERGE+INSERTION SORT 349349797

Эксперимент подтвердил преимущество гибридного алгоритма MERGE+INSERTION SORT над стандартной реализацией merge sort
 Оптимальное значение порога переключения составляет 15 элементов, что обеспечивает баланс между накладных расходов рекурсии и эффективностью insertion sort